"Heaven's Light is Our Guide"

OBE Evaluation and Grading Webapp

Submitted By

Md. Sanzid Alam Sahil	(2310010)	Mustakim Al Siyam	(2310047)
Mahathir Mohammad Mahi	(2310024)	Md. Azaz Arfin Khan	(2310060)
Tahsin Maria	(2310034)	Shuvo Chakma	(2310061)

Software Development Project-I (ECE-2100)

Supervisors:

Md. Omaer Faruq Goni
Assistant Professor
Department of Electrical & Computer Engineering, RUET

Oishi Jyoti
Assistant Professor
Department of Electrical & Computer Engineering, RUET

Rajshahi University of Engineering & Technology
Faculty of Electrical & Computer Engineering
Department of Electrical & Computer Engineering
Rajshahi-6204

April 2026

Acknowledgment

All praise and gratitude belong to the Almighty for granting us the strength, patience, and perseverance to successfully complete this project within the stipulated time frame.

We would like to express our profound and sincere gratitude to our esteemed supervisors, **Md. Omaer Faruq Goni**, Assistant Professor, Department of Electrical & Computer Engineering, RUET, and **Oishi Jyoti**, Assistant Professor, Department of Electrical & Computer Engineering, RUET, for their invaluable guidance, continuous encouragement, constructive criticism, and unwavering support throughout every phase of this project. Their insightful suggestions and technical expertise were instrumental in shaping the direction and quality of this work. Without their mentorship, this project would not have achieved its present form.

We are deeply indebted to the **Department of Electrical & Computer Engineering** at the Rajshahi University of Engineering & Technology for providing us with a conducive academic environment, access to computational resources, and a robust curriculum that fostered our skills in software engineering and web development. The laboratory facilities and internet infrastructure made available by the department were critical to the development and testing phases of this project.

We extend our heartfelt thanks to all the faculty members of the ECE Department who, through their dedicated teaching of courses in programming, data structures, database management, and computer networks, equipped us with the foundational knowledge required to undertake a project of this scope and complexity.

We also acknowledge the contributions of the open-source software community — particularly the maintainers and contributors of React, Node.js, Express.js, MongoDB, Firebase, and the broader JavaScript ecosystem — whose freely available tools, libraries, and documentation made it possible to build a modern, production-grade web application.

Special thanks go to our classmates and peers in the 2023 Series of the ECE Department, who generously participated in user acceptance testing, provided constructive feedback on the application's interface and functionality, and served as the primary test cohort for the student-facing modules of the platform.

Finally, we express our deepest appreciation to our families for their constant moral support, understanding, and encouragement throughout the duration of this project.

Their patience during the long hours of development and debugging sessions did not go unnoticed.

The Authors
April 2026
Rajshahi, Bangladesh

Abstract

Outcome-Based Education (OBE) has emerged as the predominant pedagogical framework for engineering programs worldwide, mandated by international accreditation bodies such as the Washington Accord and adopted domestically by the Board of Accreditation for Engineering and Technical Education (BAETE) in Bangladesh. Despite this widespread adoption, the practical implementation of OBE at the institutional level — particularly the mapping of Course Outcomes (COs) to Program Outcomes (POs), the computation of attainment levels, and the generation of Continuous Quality Improvement (CQI) documentation — remains a labor-intensive, error-prone, and largely manual process at most Bangladeshi engineering universities, including the Rajshahi University of Engineering & Technology (RUET).

This project presents the design, development, and deployment of a comprehensive, web-based OBE Evaluation and Grading platform built specifically for RUET's Department of Electrical & Computer Engineering. The system is architected as a full-stack MERN (MongoDB, Express.js, React, Node.js) application that automates the entire OBE workflow — from course creation and student enrollment through mark entry, attendance tracking, CO-PO attainment calculation, and institutional feedback collection.

The platform implements a robust Role-Based Access Control (RBAC) system supporting four distinct user roles: **Central Administrator**, **Department Administrator**, **Teacher**, and **Student**. Each role is provided with a specialized, role-appropriate dashboard and feature set. The Central Administrator oversees university-wide data including departments, courses, teachers, and bulk student management. The Department Administrator manages course assignments, teacher allocations, and reviews aggregated feedback analytics through a dedicated Course Review Hub. Teachers interact with a streamlined interface that abstracts the complexity of OBE calculations — they input attendance, create assessments mapped to specific COs, enter marks via spreadsheet-like grids, and generate automated Evaluation Reports comprising both traditional marksheets and OBE attainment metrics. Students access a transparent academic tracking portal showing their personal course progress, raw marks, attendance records, individual CO/PO attainment percentages, and an anonymous course feedback submission mechanism.

The backend employs a MongoDB document database with Mongoose ODM, implementing 13 interrelated data models (User, Department, Course, ClassInstance,

Assessment, Enrollment, Feedback, InstructorReport, Notice, NoticeRead, SectionCR, Series, CourseAdvisor) connected through 13 RESTful API route modules. Authentication is handled via Firebase Authentication with JWT-based session management. The frontend is built with React 19, Redux Toolkit for state management, React Router v7 for navigation, TailwindCSS v4 for styling, and Vite v6 as the build tool. The application supports both light and dark themes, is installable as a Progressive Web App (PWA), and is deployed on Netlify with the backend hosted separately.

Key technical contributions include: (1) an automated OBE Calculation Engine that computes Best-of-N averaging for class tests, KPI-threshold-based attainment percentages, and class-level CO/PO attainment metrics; (2) a comprehensive analytics service providing 25+ pre-computed statistical measures for institutional decision-making; (3) a dual-feedback system capturing both anonymous student evaluations and structured instructor experience reports for CQI documentation; and (4) an Excel/PDF export pipeline enabling offline record-keeping.

The system has been successfully deployed and tested with representative data from the ECE Department's 2023 Series, demonstrating significant reductions in manual calculation effort, elimination of formula-based errors inherent in spreadsheet workflows, and improved transparency in the OBE evaluation process for all stakeholders.

Keywords: Outcome-Based Education, MERN Stack, Web Application, Course Outcome, Program Outcome, OBE Attainment, Role-Based Access Control, RUET, Educational Technology, Continuous Quality Improvement.

Contents

Acknowledgment **ii**

Abstract **iv**

1 Introduction **1**

1.1 Introduction 1

1.2 Problem Statement 3

1.3 Project Objectives & Outcomes 4

1.4 Significance of the Study 5

1.5 Scope of the Project 7

2 Requirements & Tools **10**

2.1 Introduction 10

2.2 Hardware Requirements 10

2.3 Software Requirements 12

2.3.1 Functional Requirements 12

2.3.2 Non-Functional Requirements 15

2.4 Programming Languages & Frameworks 15

2.4.1 JavaScript (ECMAScript 2024) 15

2.4.2 React 19 (Frontend Framework) 15

2.4.3 Node.js & Express.js 5 (Backend Runtime & Framework) . . . 16

2.4.4 MongoDB & Mongoose 9 (Database) 16

2.4.5 Firebase Authentication 17

2.4.6 Redux Toolkit (State Management) 17

2.4.7 TailwindCSS 4 (Styling) 17

2.4.8 Vite 6 (Build Tool) 17

2.4.9 Additional Libraries 18

2.5 Development Tools 18
2.6 System Architecture Overview 18
2.6.1 Architecture Diagram 18
2.6.2 Data Flow 18
2.6.3 Deployment Architecture 20

3 System Implementation **21**
3.1 Introduction 21
3.2 Module-Wise Implementation 21
3.2.1 Database Design 21
3.2.2 Authentication & Role-Based Access Control 24
3.2.3 Backend API Design 26
3.2.4 OBE Calculation Engine 27
3.3 Frontend Feature Implementation 29
3.3.1 Central Admin Panel 29
3.3.2 Department Admin Panel 30
3.3.3 Teacher Panel 30
3.3.4 Student Panel 32
3.3.5 Common Components 33

4 Project Management & Participation **34**
4.1 Development Methodology 34
4.1.1 Agile Principles Applied 34
4.1.2 Version Control Workflow 34
4.2 Timeline / Gantt Chart 35
4.3 Team Roles & Contributions 37
4.4 Attendance & Participation 37

References **40**

Appendices **42**

A Source Code & GitHub Repository **43**

B Application Screenshots **49**

C User Manual **55**

List of Figures

2.1 Three-Tier System Architecture Diagram 19

4.1 Project Development Gantt Chart (Mar 18 – Apr 25, 2026) 35

B.1 Login Page — RUET OBE Evaluation System 49

B.2 Central Admin Dashboard — University Overview 50

B.3 Department Admin — Feedback Analytics Dashboard 50

B.4 Teacher Dashboard — Course Overview 51

B.5 Teacher Course Hub — Quick Actions 52

B.6 Attendance Taking Interface — Spreadsheet-Style Grid 52

B.7 Evaluation Report — General Marksheet with Grade Computation . 53

B.8 Student Portal — Dashboard with Running Courses and PO Attainment 54

B.9 Student Course Page — Overview with Resource Cards 54

List of Tables

1.1 Project Objectives Mapped to Course Outcomes 5
1.2 Course Outcome to Program Outcome Mapping Matrix 6

2.1 Development Environment Hardware Specifications 11
2.2 Production Deployment Infrastructure 11
2.3 Functional Requirements by User Role 12
2.4 Non-Functional Requirements 14
2.5 Frontend Framework Comparison 15
2.6 Database Technology Comparison 16
2.7 Additional Libraries and Their Purpose 18
2.8 Development Tools and Utilities 19
2.9 Deployment Configuration 20

3.1 Summary of All 13 MongoDB Collections 24
3.2 Backend API Route Modules 26
3.3 Sample Assessment Data for a Theory Course Student 29
3.4 Shared Frontend Components 33

4.1 Development Milestone Timeline 36
4.2 Team Member Roles and Primary Contributions 38
4.3 Contribution Distribution by System Component (%) 39
4.4 Project Meeting Attendance Record 39

C.1 Central Admin Feature Guide 56
C.2 Department Admin Feature Guide 56
C.3 Teacher Feature Guide 57
C.4 Student Feature Guide 58
C.5 Common Issues and Solutions 58

Chapter 1

Introduction

1.1 Introduction

The landscape of engineering education has undergone a fundamental paradigm shift over the past three decades, transitioning from traditional content-driven pedagogical models to Outcome-Based Education (OBE). First articulated comprehensively by William Spady in 1994 [1], OBE represents a learner-centered philosophy that organizes the entire educational system around clearly defined outcomes — the knowledge, skills, and attitudes that students are expected to demonstrate upon completion of their program of study. Unlike traditional approaches that emphasize the duration and content of instruction, OBE focuses on what students can actually *do* as a result of their educational experience.

The adoption of OBE in engineering education has been accelerated by international accreditation frameworks, most notably the Washington Accord, which established a set of twelve Graduate Attributes (GAs) that serve as the benchmark for engineering program quality across signatory nations [2]. These Graduate Attributes — encompassing engineering knowledge, problem analysis, design and development of solutions, investigation, modern tool usage, the engineer and society, environment and sustainability, ethics, individual and teamwork, communication, project management and finance, and lifelong learning — provide a universal vocabulary for describing engineering competence.

In Bangladesh, the Board of Accreditation for Engineering and Technical Education (BAETE), operating under the Institution of Engineers, Bangladesh (IEB), has adopted the Washington Accord framework and mandated OBE implementation across all accredited engineering programs [3]. This mandate requires that each course within an engineering program defines specific Course Outcomes (COs), maps these COs to the broader Program Outcomes (POs) derived from the Washington Accord's Graduate Attributes, and systematically measures student attainment of these outcomes through a combination of continuous and summative assessments.

The Rajshahi University of Engineering & Technology (RUET), established in 1964 as one of the premier engineering institutions in Bangladesh, has been actively im-

plementing OBE across its departments. The Department of Electrical & Computer Engineering (ECE), one of RUET's flagship departments, follows a rigorous OBE evaluation process that involves:

1. Defining 3–6 Course Outcomes (COs) for each course, mapped to relevant Program Outcomes (POs).
2. Creating assessments (Class Tests, Assignments, Semester Finals) with individual questions or components mapped to specific COs.
3. Computing individual student attainment percentages for each CO based on their performance in mapped assessments.
4. Determining class-level attainment by calculating the percentage of students exceeding a predefined Key Performance Indicator (KPI) threshold (typically 40% or 60%).
5. Aggregating CO attainment data to derive PO attainment at the course and program levels.
6. Collecting bidirectional feedback — student feedback on course delivery and instructor self-evaluation reports — to support Continuous Quality Improvement (CQI).

This project, the **OBE Evaluation and Grading Webapp**, was conceived to address the critical gap between the theoretical requirements of OBE and the practical tools available to implement it. By building a centralized, web-based platform using the MERN (MongoDB, Express.js, React, Node.js) technology stack, this project aims to automate the entire OBE workflow, eliminate manual calculation errors, and provide real-time visibility into educational outcomes for all stakeholders — administrators, teachers, and students alike.

The significance of this work extends beyond mere automation. In the broader context of Bangladesh's aspiration to achieve international accreditation parity for its engineering programs, tools that systematically implement and document OBE processes are essential. The University Grants Commission (UGC) of Bangladesh has emphasized the importance of Institutional Quality Assurance Cells (IQACs) and data-driven quality monitoring [4], making platforms like the one developed in this project increasingly relevant to the national educational technology landscape.

1.2 Problem Statement

Despite the mandatory adoption of OBE in engineering programs across Bangladesh, the practical implementation of OBE evaluation at RUET — and at most Bangladeshi engineering institutions — remains entrenched in manual, spreadsheet-based workflows that are fundamentally inadequate for the complexity and scale of the task. The following critical problems have been identified through direct observation of the existing workflow at RUET's ECE Department:

1. Excel-Dependent Calculation Workflow

The current OBE evaluation process relies heavily on pre-formatted Microsoft Excel templates. These templates contain complex, multi-layered formulas for computing CO attainment, PO attainment, and KPI compliance. Each course section requires its own dedicated spreadsheet, and teachers must manually replicate formulas when setting up new courses. This approach is error-prone — a single misplaced cell reference or accidental formula deletion can silently corrupt an entire course's OBE calculations. Moreover, the formulas in these templates have evolved organically over several academic cycles, resulting in inconsistencies between different teachers' spreadsheets even for the same course.

2. Fragmented Data Management

Student enrollment data, attendance records, assessment marks, CO-PO mappings, and feedback responses exist in disparate, disconnected locations — separate Excel files on individual teachers' personal computers, paper-based attendance registers, and ad-hoc Google Forms for feedback collection. There is no centralized repository for academic data, making it extremely difficult for department administrators to obtain a consolidated view of OBE compliance across all courses. When teachers change or retire, institutional knowledge embedded in their personal spreadsheet configurations is often lost.

3. Lack of Real-Time Visibility

Students have no mechanism to view their own OBE attainment status during a running semester. They receive only their raw marks — often with significant delays — and have no understanding of how their performance maps to Course Outcomes or Program

Outcomes. This opacity undermines one of OBE's core principles: empowering learners to take ownership of their educational outcomes.

4. Inconsistent Feedback Mechanisms

Student course feedback is collected through various ad-hoc methods (paper forms, Google Forms, or not at all), with no standardized question set, no systematic analysis, and no formal mechanism for feeding the results back into the curriculum improvement process. Similarly, instructor self-evaluation reports — a critical component of CQI documentation — are often submitted as free-form documents with no consistent structure, making aggregation and comparison across courses difficult.

5. Accreditation Documentation Burden

Preparing for BAETE accreditation visits requires the compilation of extensive OBE documentation — CO-PO mapping matrices, attainment charts, feedback summaries, and CQI action items — from multiple courses across multiple semesters. Under the current manual system, this compilation process is enormously time-consuming and represents a significant burden on department administrators, often requiring weeks of dedicated effort.

6. Scalability Constraints

As RUET's student population grows and new courses are added to the curriculum, the manual OBE workflow becomes increasingly unsustainable. A system that scales with the institution's needs is no longer a convenience — it is a necessity.

These problems collectively demonstrate the need for an integrated, automated, web-based solution that can streamline the OBE evaluation workflow, centralize academic data management, and provide transparent, real-time access to educational outcome metrics for all stakeholders.

1.3 Project Objectives & Outcomes

The primary objective of this project is to design, develop, and deploy a comprehensive web-based platform that automates the Outcome-Based Education evaluation and grading process for the ECE Department at RUET. The specific objectives, each linked

to the Course Outcomes (COs) of ECE-2100 (Software Development Project-I), are enumerated below:

Table 1.1: Project Objectives Mapped to Course Outcomes

Obj.	Objective Description	Mapped CO
O1	Design and implement a role-based web application with four distinct user panels (Central Admin, Department Admin, Teacher, Student) using the MERN stack, demonstrating proficiency in full-stack software development.	CO1
O2	Develop an automated OBE Calculation Engine that computes Course Outcome (CO) and Program Outcome (PO) attainment levels from raw assessment data, replacing manual Excel-based workflows with algorithmic precision.	CO2
O3	Implement a centralized database system using MongoDB that stores and manages all academic data — enrollments, assessments, attendance, marks, feedback — in a structured, queryable, and scalable manner.	CO3
O4	Create an intuitive, responsive user interface with spreadsheet-like data entry grids, interactive dashboards, and automated report generation capabilities that minimize the learning curve for faculty users.	CO4
O5	Establish a bidirectional feedback system comprising anonymous student course evaluations and structured instructor experience reports, integrated with departmental analytics for Continuous Quality Improvement (CQI).	CO5

Furthermore, the following table maps the Course Outcomes of ECE-2100 to the Program Outcomes (POs) of the ECE undergraduate program at RUET:

1.4 Significance of the Study

The development of the OBE Evaluation and Grading Webapp carries significant implications across multiple dimensions of engineering education at RUET and, by extension, the broader landscape of higher education in Bangladesh.

For Teachers

The platform eliminates the most time-consuming and error-prone aspects of the OBE evaluation workflow. Teachers no longer need to maintain individual Excel

Table 1.2: Course Outcome to Program Outcome Mapping Matrix

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	2	3	1	3	–	–	–	2	–	2	1
CO2	3	3	2	3	2	–	–	–	1	–	–	2
CO3	2	2	3	2	3	–	–	–	–	–	1	1
CO4	1	1	3	1	3	–	–	1	2	3	2	–
CO5	1	–	2	2	1	1	–	2	3	2	–	3

Mapping levels: 3 = High correlation, 2 = Medium correlation, 1 = Low correlation, – = No direct correlation

templates with complex, fragile formulas. Instead, they interact with an intuitive web interface that abstracts all computational complexity behind simple data entry forms. The spreadsheet-like grading interface mirrors the familiar Excel experience while ensuring that all underlying OBE calculations are performed automatically and consistently. The automated Evaluation Report feature generates both traditional marksheets and OBE attainment metrics at the click of a button, reducing report preparation time from hours to seconds. The Instructor Experience Report module provides a structured framework for CQI documentation, ensuring compliance with accreditation requirements without imposing additional cognitive burden on faculty.

Based on conservative estimates, the platform reduces the per-course OBE evaluation effort from approximately 8–12 hours of manual Excel work to less than 1 hour of data entry, representing a **time savings of approximately 85–90%**.

For Students

For the first time, students at RUET gain transparent, real-time access to their own OBE attainment data. Through the Student Panel, they can view not only their raw assessment marks and attendance records but also their individual CO and PO attainment percentages, providing them with a clear understanding of their strengths and areas for improvement relative to the curriculum's intended learning outcomes. The anonymous feedback mechanism empowers students to contribute meaningfully to course improvement without fear of academic reprisal.

For Department Administrators

The Course Review Hub provides department administrators with an unprecedented consolidated view of educational quality across all courses. By aggregating both student feedback and instructor self-evaluations, the platform enables data-driven

decision-making for curriculum revisions, resource allocation, and faculty development. The centralized data architecture eliminates the need for manual data aggregation from disparate spreadsheets, significantly streamlining accreditation preparation.

For Institutional Accreditation

The platform generates OBE documentation artifacts — CO-PO mapping matrices, attainment charts, feedback summaries — in a standardized, queryable format that aligns directly with BAETE accreditation requirements. This positions the ECE Department to demonstrate systematic OBE implementation with empirical evidence, a critical factor in successful accreditation outcomes.

For the Broader Educational Technology Landscape

This project contributes a reusable, open-source[1] template for OBE automation that can be adapted by other departments at RUET or by other engineering institutions in Bangladesh facing similar challenges. The MERN-stack architecture, modular codebase, and comprehensive API design make the system highly extensible and adaptable to different institutional contexts.

1.5 Scope of the Project

This section delineates the boundaries of the project, clearly specifying what is included within the scope of the current implementation and what has been intentionally deferred for future development.

Included in Scope

- **Four-Role RBAC System:** Complete implementation of Central Administrator, Department Administrator, Teacher, and Student panels with role-appropriate feature sets and data isolation.
- **Theory Course Evaluation:** Full support for the ECE Department's theory course mark distribution — 4 Class Tests (Best 3 of 4 averaging to 20 marks), Attendance (10 marks), Assignment (10 marks), and Semester Final Exam (60 marks) with Section A/B question-level CO mapping.

[1] Source code available at: https://github.com/azazarfin/obe-webapp-demo

- **Sessional/Lab Course Evaluation:** Full support for sessional course mark distribution — Quiz (20 marks), Performance/Report (45 marks), Attendance (10 marks), and Lab Final/Viva (25 marks) with CO mapping.
- **OBE Calculation Engine:** Automated computation of CO attainment (individual and class-level), PO attainment via CO-PO mapping matrices, and KPI-threshold-based compliance checking.
- **Attendance Management:** Date-wise attendance tracking with present/absent status, attendance percentage calculations, and mark-based attendance scoring.
- **Feedback System:** Standardized 6-question student feedback form with Likert-scale responses, anonymous submission, aggregated analytics with visualization, and Excel export capability.
- **Instructor Experience Reports:** Structured CQI documentation with evaluation attributes (Likert-scale ratings) and written analysis sections covering syllabus, teaching methods, resources, and assessment processes.
- **Analytics & Reporting:** Automated generation of General Marksheets, OBE Attainment Reports, feedback summary dashboards, and departmental Course Review Hub with gap analysis.
- **Data Management:** Bulk student upload, course and department CRUD operations, series management, course advisor assignment, and notification system.
- **Deployment:** Production deployment on Netlify (frontend) with a cloud-hosted MongoDB Atlas database and Firebase Authentication.
- **Progressive Web App (PWA):** Installable on mobile and desktop devices via the browser's "Add to Home Screen" functionality.

Excluded from Scope (Future Work)

- **Native Mobile Applications:** Dedicated iOS and Android applications are not included; the PWA capability provides a mobile-friendly alternative.
- **Multi-University Support:** The current system is designed for a single university (RUET). Multi-tenant architecture for multiple institutions is deferred.

- **Integration with National Education Systems:** Integration with Bangladesh's national education management systems (e.g., UGC databases) is not implemented.
- **Advanced Analytics & Machine Learning:** Predictive analytics for student performance, automated curriculum recommendation, and machine learning-based outcome prediction are beyond the current scope.
- **Automated Exam Scheduling:** The platform does not include exam scheduling or timetable generation features.
- **Plagiarism Detection:** Integration with plagiarism detection services for assignment submissions is not included.
- **Video Conferencing:** Online lecture delivery or virtual classroom features are outside the scope.

Chapter 2

Requirements & Tools

2.1 Introduction

This chapter presents a comprehensive analysis of the requirements engineering process undertaken for the OBE Evaluation and Grading Webapp. Requirements engineering is a critical early-phase activity in software development that establishes the functional and non-functional characteristics a system must exhibit to satisfy stakeholder needs [5, 6]. The requirements for this project were elicited through a combination of direct observation of existing OBE workflows at RUET's ECE Department, analysis of the official BAETE accreditation templates, structured interviews with faculty members who regularly perform OBE evaluations, and review of the Excel-based OBE templates currently in use.

The chapter is organized as follows: Section 2.2 describes the hardware requirements for both development and deployment. Section 2.3 details the software requirements, including functional and non-functional specifications. Section 2.4 provides an in-depth analysis of the programming languages and frameworks selected for the project, with comparative justifications. Section 2.5 enumerates the development tools and utilities used throughout the project lifecycle. Finally, Section 2.6 presents the system architecture overview with supporting diagrams.

2.2 Hardware Requirements

The hardware requirements for this project are categorized into two groups: development environment hardware (used by the team during the implementation phase) and deployment/production hardware (the server infrastructure on which the application runs in production).

Client-Side Requirements: End users (teachers, students, administrators) require only a modern web browser (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+) and an internet connection. The application is fully responsive and supports devices ranging from desktop monitors to mobile phones.

Table 2.1: Development Environment Hardware Specifications

Component	**Specification**
Processor	Intel Core i5 (8th generation or higher) / AMD Ryzen 5 equivalent
RAM	Minimum 8 GB DDR4 (16 GB recommended for concurrent frontend/backend development)
Storage	Minimum 256 GB SSD (for fast build times and hot-reload performance)
Display	14–15.6 inch display with 1920×1080 resolution (for side-by-side code/browser layout)
Network	Stable broadband internet connection (minimum 10 Mbps) for NPM package installation, API testing, and cloud database access
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+ (cross-platform development supported)

Table 2.2: Production Deployment Infrastructure

Component	**Specification / Service**
Frontend Hosting	Netlify (CDN-backed static hosting with automatic HTTPS, continuous deployment from Git)
Backend Hosting	Cloud-based Node.js runtime (auto-scaling, HTTPS)
Database Server	MongoDB Atlas (M0 free tier for development; M10+ recommended for production) — cloud-managed, auto-scaling replica set
Authentication	Firebase Authentication (Google Cloud infrastructure — globally distributed, auto-scaling)
DNS & SSL	Managed by Netlify (automatic Let's Encrypt SSL certificates)

2.3 Software Requirements

2.3.1 Functional Requirements

The functional requirements define the specific behaviors and features that the system must provide. These requirements are organized by user role to reflect the RBAC-based architecture of the application.

Table 2.3: Functional Requirements by User Role

ID	Role / Module	Requirement Description
FR-01	Authentication	The system shall authenticate users via Firebase Authentication using email/password credentials.
FR-02	Authentication	The system shall enforce role-based access control, restricting each user to their authorized panel and data.
FR-03	Central Admin	The system shall allow the Central Admin to create, read, update, and delete departments.
FR-04	Central Admin	The system shall allow the Central Admin to manage courses across all departments with filtering by department and semester.
FR-05	Central Admin	The system shall support bulk student upload via an Excel-like grid interface, automatically generating RUET-format student IDs and edu-mail addresses.
FR-06	Central Admin	The system shall allow the Central Admin to manage teacher profiles including name, designation, department, joining year, and teacher type (Host/Guest).
FR-07	Dept Admin	The system shall allow Department Admins to assign teachers to courses by selecting Course Code, Series, Section, and Teacher from dropdown menus.
FR-08	Dept Admin	The system shall provide a Course Review Hub displaying aggregated student feedback and instructor experience reports with gap analysis visualization.
FR-09	Teacher	The system shall allow teachers to take date-wise attendance using a spreadsheet-like interface with Present/Absent toggles.

ID	Role / Module	Requirement Description
FR-10	Teacher	The system shall allow teachers to create assessments (CT, Assignment, Final, Quiz, Report, Lab Final, Custom) with CO and PO mappings.
FR-11	Teacher	The system shall allow teachers to enter student marks via a spreadsheet-like grid, with 'A' interpreted as absent (0 marks).
FR-12	Teacher	The system shall automatically generate General Marksheets showing raw scores, attendance marks, and total marks.
FR-13	Teacher	The system shall automatically generate OBE Attainment Reports showing individual and class-level CO/PO attainment percentages.
FR-14	Teacher	The system shall provide a Semester Final marking interface with Section A/B question-level CO mapping.
FR-15	Teacher	The system shall allow teachers to submit structured Instructor Experience Reports.
FR-16	Student	The system shall display each student's personal marksheet, attendance record, and CO/PO attainment percentages.
FR-17	Student	The system shall allow students to submit anonymous course feedback via a standardized 6-question Likert-scale form.
FR-18	Student	The system shall provide a university directory allowing students to browse courses and teachers by department.
FR-19	System-wide	The system shall support both light and dark themes with persistent user preference.
FR-20	System-wide	The system shall be installable as a Progressive Web App on mobile and desktop devices.

Table 2.4: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API response time $\leq$ 200ms for standard CRUD operations under normal load.
NFR-02	Performance	Frontend initial load time $\leq$ 3 seconds on a standard broadband connection.
NFR-03	Scalability	The system shall handle concurrent access by at least 100 simultaneous users without degradation.
NFR-04	Security	All API endpoints shall be protected by JWT-based authentication middleware.
NFR-05	Security	Rate limiting shall be enforced on authentication endpoints (maximum 10 attempts per 15-minute window).
NFR-06	Security	CORS policies shall restrict API access to authorized frontend origins only.
NFR-07	Usability	The application shall be fully responsive across desktop (1920×1080), tablet (768×1024), and mobile (375×667) viewports.
NFR-08	Reliability	The system shall maintain 99.5% uptime through cloud-hosted infrastructure with automatic failover.
NFR-09	Maintainability	The codebase shall follow modular architecture with separation of concerns (routes, models, services, utilities).
NFR-10	Compatibility	The frontend shall be compatible with the latest two major versions of Chrome, Firefox, Safari, and Edge.

2.3.2 Non-Functional Requirements

2.4 Programming Languages & Frameworks

The technology stack for this project was selected after careful evaluation of multiple alternatives, prioritizing developer productivity, ecosystem maturity, real-time performance, and alignment with industry best practices for modern web application development.

2.4.1 JavaScript (ECMAScript 2024)

JavaScript serves as the unified programming language for both the frontend and backend of this application. This "full-stack JavaScript" approach eliminates the context-switching overhead associated with using different languages for client and server code, enables code sharing between layers (e.g., validation logic), and leverages the largest package ecosystem in the world (NPM, with over 2.5 million packages).

2.4.2 React 19 (Frontend Framework)

React, developed and maintained by Meta Platforms, is a declarative, component-based JavaScript library for building user interfaces [7]. React 19, the version used in this project, introduces significant performance improvements including automatic batching, concurrent rendering, and the new use hook for streamlined data fetching.

Why React over alternatives:

Table 2.5: Frontend Framework Comparison

Criterion	**React**	**Angular**	**Vue.js**
Learning Curve	Moderate (JSX, hooks)	Steep (TypeScript, RxJS, DI)	Gentle (template syntax)
Ecosystem	Largest (Meta, massive community)	Google-backed, enterprise	Growing community
Performance	Excellent (virtual DOM, concurrent mode)	Good (Ivy compiler)	Excellent (reactivity system)
Job Market	Highest demand	Strong enterprise	Growing
Flexibility	High (library, not framework)	Low (opinionated)	Medium

React was selected for its superior ecosystem, component reusability model, and the availability of mature complementary libraries (Redux Toolkit, React Router, Lucide icons).

2.4.3 Node.js & Express.js 5 (Backend Runtime & Framework)

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a web browser [8]. Express.js 5, the latest major version of the most popular Node.js web framework [9], provides:

- Minimal, unopinionated routing with middleware-based architecture
- Built-in async error handling (new in Express 5)
- High throughput for I/O-bound operations via Node.js's event-driven, non-blocking architecture
- Seamless integration with MongoDB via Mongoose ODM

2.4.4 MongoDB & Mongoose 9 (Database)

MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like BSON documents [10]. Mongoose 9 [11] provides schema-based modeling, validation, middleware (pre/post hooks), and query building on top of MongoDB's native driver.

Why MongoDB over relational databases:

Table 2.6: Database Technology Comparison

Criterion	**MongoDB (NoSQL)**	**PostgreSQL (SQL)**
Schema Flexibility	Flexible schemas — ideal for evolving models	Rigid schemas require migrations
Data Model Fit	Natural fit for nested data (enrollment marks, attendance arrays)	Requires JOIN-heavy normalized tables
Horizontal Scaling	Built-in sharding	Limited horizontal scaling
JSON Compatibility	Native BSON/JSON — zero serialization overhead	Requires JSON ↔ row mapping
Development Speed	Faster iteration with schema changes	Slower (migration scripts needed)

MongoDB was chosen because the application's data models (particularly Enrollment with nested attendance arrays and marks arrays) naturally map to document structures, and the schema flexibility accelerated the iterative development process.

2.4.5 Firebase Authentication

Firebase Authentication [12] is a cloud-hosted identity service by Google that provides email/password authentication, multi-factor authentication, and social login capabilities. In this project, Firebase handles user authentication on the client side, while the backend verifies Firebase ID tokens using the Firebase Admin SDK to issue application-specific JWTs [13].

2.4.6 Redux Toolkit (State Management)

Redux Toolkit [14] is the official, opinionated toolset for Redux — the most widely adopted state management library for React applications. It is used in this project to manage global application state including:

- Authentication state (current user, role, JWT token)
- Sidebar navigation state (expanded/collapsed groups, active page)
- Theme preference (light/dark mode)

2.4.7 TailwindCSS 4 (Styling)

TailwindCSS [15] is a utility-first CSS framework that enables rapid UI development through composable utility classes. Version 4 introduces a redesigned engine with significant performance improvements and first-class support for the `@tailwindcss/vite` plugin used in this project.

2.4.8 Vite 6 (Build Tool)

Vite [16] is a next-generation frontend build tool that provides near-instantaneous Hot Module Replacement (HMR) during development and optimized production builds using Rollup. Its native ES module support eliminates the need for bundling during development, resulting in sub-second page reload times.

2.4.9 Additional Libraries

Table 2.7: Additional Libraries and Their Purpose

Library	Version	Purpose
React Router	v7	Declarative client-side routing with nested layouts [17]
Lucide React	v0.577	Icon library providing 1,500+ SVG icons [18]
ExcelJS	v4.4	Client-side Excel file generation for feedback export
jsPDF + AutoTable	v4.2 / v5.0	Client-side PDF generation for reports
bcryptjs	v3.0	Password hashing for local authentication fallback
express-rate-limit	v8.3	API rate limiting middleware
CORS	v2.8	Cross-Origin Resource Sharing configuration
dotenv	v17.3	Environment variable management

2.5 Development Tools

2.6 System Architecture Overview

The OBE Evaluation and Grading Webapp follows a **three-tier client-server architecture** that separates concerns into three distinct layers: the Presentation Tier (frontend), the Application Tier (backend API), and the Data Tier (database and authentication services).

2.6.1 Architecture Diagram

2.6.2 Data Flow

The typical data flow through the system follows this sequence:

1. **Authentication:** The user logs in via the React frontend, which communicates with Firebase Authentication to obtain an ID token. This token is sent to the backend, which verifies it using the Firebase Admin SDK and issues a session JWT.

Table 2.8: Development Tools and Utilities

Tool	Purpose
Visual Studio Code	Primary code editor with extensions for JavaScript, React, ESLint, and Git integration
Git & GitHub	Distributed version control system [19] and remote repository hosting for collaborative development
Postman	API testing and documentation tool for validating all 13 REST API route modules
MongoDB Compass	GUI for MongoDB database visualization, query building, and schema analysis
Chrome DevTools	Browser-based debugging, performance profiling, network analysis, and responsive design testing
Netlify	Continuous deployment platform for frontend hosting [20] with automatic builds from Git pushes
Firebase Console	Authentication user management, usage analytics, and security rule configuration
ESLint	Static code analysis for identifying and fixing JavaScript/React code quality issues
Nodemon	Development utility that automatically restarts the Node.js server upon file changes
NPM	Node Package Manager for dependency management (frontend and backend)

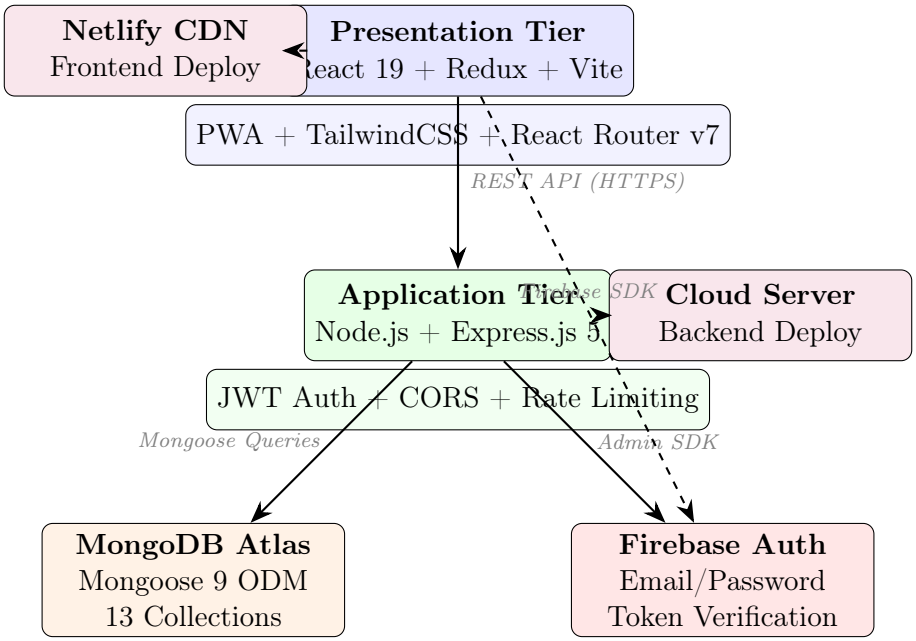

Figure 2.1: Three-Tier System Architecture Diagram

2. **API Request:** The frontend sends an HTTPS request to the Express.js backend, including the JWT in the Authorization header. The backend's authentication middleware validates the token and extracts the user's role.

3. **Authorization:** Role-based middleware checks whether the user's role has permission to access the requested resource.

4. **Data Processing:** The route handler performs business logic (e.g., OBE attainment calculation) and interacts with MongoDB via Mongoose models.

5. **Response:** The processed data is returned to the frontend as a JSON response, where it is rendered in the appropriate UI component.

2.6.3 Deployment Architecture

Table 2.9: Deployment Configuration

Component	Service	Configuration
Frontend	Netlify	Auto-deploy from `main` branch, build command: `npm run build`, publish dir: `dist/`
Backend	Cloud Server	Node.js runtime, environment variables for MongoDB URI, Firebase credentials, JWT secret
Database	MongoDB Atlas	Cloud-managed replica set, IP whitelisting, automated backups
Auth	Firebase	Email/password provider enabled, authorized domains configured
Domain	Netlify	`obe-webapp-demo.netlify.app` with automatic HTTPS

Chapter 3

System Implementation

3.1 Introduction

This chapter presents a detailed technical account of the implementation of the OBE Evaluation and Grading Webapp. The system was built incrementally over a six-week development cycle using an Agile methodology, with each sprint delivering a functional, testable vertical slice of functionality. The implementation is organized into two broad categories: (1) backend modules covering the database schema, authentication system, RESTful API, and OBE Calculation Engine; and (2) frontend feature implementations covering all four role-based panels and shared components.

The source code is hosted publicly at `https://github.com/azazarfin/obe-webapp-demo` and the live deployment is available at `https://obe-webapp-demo.netlify.app`. The project comprises approximately **13 backend route modules**, **13 Mongoose data models**, **6 backend service files**, and over **40 React component/page files**.

3.2 Module-Wise Implementation

3.2.1 Database Design

The application uses MongoDB as its database, with Mongoose ODM providing schema definition, validation, and pre/post save hooks. The data architecture consists of 13 collections, designed to minimize redundancy while supporting efficient querying for OBE calculations.

Core Data Models

User Model — The central identity record for all four roles. A single `User` collection stores Central Admins, Department Admins, Teachers, and Students, differentiated by the `role` field.

```
const userSchema = new mongoose.Schema({
  name:       { type: String, required: true },
```

```
  email:      { type: String, required: true, unique: true },
  role:       { type: String,
                enum: ['CENTRAL_ADMIN','DEPT_ADMIN','TEACHER','STUDENT'],
                required: true },
  department: { type: mongoose.Schema.Types.ObjectId, ref: 'Department' },
  rollNumber: { type: String },   // RUET format: 2310060
  series:     { type: Number },   // e.g., 2023
  section:    { type: String, enum: ['N/A','A','B','C','D','E'] },
  designation:{ type: String },   // for Teachers
  joiningYear:{ type: Number },
  teacherType:{ type: String, enum: ['Host','Guest'] },
  firebaseUid:{ type: String }
}, { timestamps: true });
```

Listing 3.1: User Model Schema (backend/models/User.js)

ClassInstance Model — The core operational unit linking a `Course`, a `series`, a `section`, and one or more `Teachers`. Each `ClassInstance` also stores the CO-PO mapping matrix applicable to that specific run of the course.

```
const classInstanceSchema = new mongoose.Schema({
  course:   { type: ObjectId, ref: 'Course', required: true },
  series:   { type: Number, required: true },
  section:  { type: String, enum: ['N/A','A','B','C','D','E'], default:'N/A'},
  teacher:  { type: ObjectId, ref: 'User', required: true },
  teachers: [{ type: ObjectId, ref: 'User' }],
  status:   { type: String, enum: ['Running','Finished'], default:'Running'},
  coPoMapping: [{
    co:          { type: String, required: true },   // "CO1"
    po:          [{ type: String, required: true }], // ["PO1","PO3"]
    description: { type: String, default: '' }
  }],
  feedbackQuestions: { type: [String], default: [] },
  feedbackPublished: { type: Boolean, default: false }
}, { timestamps: true });
```

Listing 3.2: ClassInstance Model Schema

Assessment Model — Represents a single evaluated activity (CT, Assignment, Final exam question, Quiz, etc.) associated with a `ClassInstance` and mapped to a specific `CO`.

```
const assessmentSchema = new mongoose.Schema({
  classInstance: { type: ObjectId, ref: 'ClassInstance', required: true },
  title:         { type: String, required: true },  // "CT-1", "Final-Q2a"
  type:          { type: String, required: true,
```

```
                    enum: ['CT','Assignment','Final','Quiz','Report',
                           'LabFinal','Presentation','Viva','Custom'] },
  totalMarks:    { type: Number, required: true },
  mappedCO:      { type: String },         // "CO2"
  mappedPOs:     { type: [String], default: [] },
  typeLabel:     { type: String },
  assessmentDate:{ type: Date },
  finalPart:     { type: String, enum: ['A','B'] },
  questionNo:    { type: String },
  createdBy:     { type: ObjectId, ref: 'User' }
}, { timestamps: true });
```

Listing 3.3: Assessment Model Schema

Enrollment Model — Links a `Student` to a `ClassInstance`, storing their attendance records and assessment marks as nested arrays.

```
const enrollmentSchema = new mongoose.Schema({
  student:       { type: ObjectId, ref: 'User', required: true },
  classInstance: { type: ObjectId, ref: 'ClassInstance', required: true },
  type:          { type: String, enum: ['regular','irregular'], default:'
    regular'},
  status:        { type: String, enum: ['active','hidden'], default:'active'},
  attendanceRecord: [{
    date:    { type: Date, required: true },
    status:  { type: String, enum: ['Present','Absent'], required: true },
    takenBy: { type: ObjectId, ref: 'User' }
  }],
  marks: [{
    assessment: { type: ObjectId, ref: 'Assessment', required: true },
    rawScore:   { type: Number, required: true }
  }]
}, { timestamps: true });
```

Listing 3.4: Enrollment Model Schema

Supporting Models Summary

Entity Relationship Overview

The relationships between the primary collections are as follows:

- A **Department** has many **Courses** and many **Users** (teachers and students).

Table 3.1: Summary of All 13 MongoDB Collections

Collection	Purpose
User	Identity and profile for all 4 roles
Department	University department registry
Course	Course catalogue with type (Theory/Sessional) and syllabus
ClassInstance	A specific run of a course by a teacher for a series/section
Assessment	Individual evaluated components mapped to COs
Enrollment	Student–class link with attendance and marks arrays
Feedback	Anonymous student course feedback (Likert ratings)
InstructorReport	Teacher's end-of-semester CQI self-evaluation
Notice	Departmental announcements visible to students
NoticeRead	Tracks which students have read each notice
SectionCR	Course Representative assignments per class instance
Series	Academic year (series) registry
CourseAdvisor	Links an advisor teacher to a student series

- A **Course** + **Series** + **Section** + **Teacher** combination creates a **ClassInstance**.
- A **ClassInstance** has many **Assessments** and many **Enrollments**.
- An **Enrollment** embeds a student's attendance records and marks for that class.
- A **ClassInstance** has one optional **InstructorReport** and many **Feedbacks**.

3.2.2 Authentication & Role-Based Access Control

The authentication system uses a two-layer approach: Firebase Authentication handles credential verification on the client side, and the backend issues and validates application-specific JWTs for all API requests.

```
const verifyToken = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Unauthorized: No token provided' });
  }

  const token = authHeader.split(' ')[1];

```

```
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded;  // { uid, role, email, userId }
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Unauthorized: Invalid token' });
  }
};

const requireRole = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({ error: 'Forbidden: Insufficient permissions'
    });
  }
  next();
};
```

Listing 3.5: JWT Verification Middleware (authMiddleware.js)

Every protected API route applies `verifyToken` followed by `requireRole(...)` with the permitted roles. For example, a teacher-only endpoint is declared as:

```
// Only TEACHER and DEPT_ADMIN may access evaluation reports
router.get(
  '/:id/evaluation',
  verifyToken,
  requireRole('TEACHER', 'DEPT_ADMIN', 'CENTRAL_ADMIN'),
  async (req, res) => {
    const data = await buildClassEvaluation(req.params.id);
    res.json(data);
  }
);
```

Listing 3.6: Role-Protected Route Example

On the frontend, the `ProtectedRoute` component reads the user's role from the Redux store and redirects unauthorized users:

```
const ProtectedRoute = ({ allowedRoles }) => {
  const { currentUser, userRole, loading } = useAuth();

  if (loading) return <LoadingSpinner />;
  if (!currentUser) return <Navigate to="/login" replace />;
  if (!allowedRoles.includes(userRole))
    return <Navigate to="/unauthorized" replace />;

  return <Outlet />;
```

```
};
```

Listing 3.7: ProtectedRoute Component

3.2.3 Backend API Design

The backend exposes a RESTful API organized into 13 Express.js router modules, all mounted under the `/api` prefix.

Table 3.2: Backend API Route Modules

Module	Mount Path	Key Operations
authRoutes	`/api/auth`	Login (Firebase token → JWT), register, get current user
departmentRoutes	`/api/departments`	CRUD for departments
courseRoutes	`/api/courses`	CRUD for courses; filter by dept/semester
userRoutes	`/api/users`	CRUD for teachers/students; bulk student upload
classInstanceRoutes	`/api/class-instances`	Create/manage class assignments; evaluation report endpoint
assessmentRoutes	`/api/assessments`	Create/update/delete assessments; bulk mark entry
enrollmentRoutes	`/api/enrollments`	Attendance taking; mark entry; roster management
feedbackRoutes	`/api/feedback`	Submit/retrieve student feedback; analytics aggregation
instructorReportRoutes	`/api/instructor-reports`	Submit/retrieve instructor experience reports
dashboardRoutes	`/api/dashboard`	Aggregated stats for admin dashboards
seriesRoutes	`/api/series`	Manage academic year (series) records
noticeRoutes	`/api/notices`	Create/publish notices; mark as read
courseAdvisorRoutes	`/api/course-advisors`	Assign/remove course advisors per series

The server applies layered security including rate-limiting on login endpoints (10 attempts per 15-minute window), CORS whitelisting, a 1 MB JSON payload cap, and a global error handler that prevents internal stack traces from leaking to clients.

3.2.4 OBE Calculation Engine

The OBE Calculation Engine automates the entire attainment pipeline previously performed in Excel, implemented in `obeEngine.js` and `analyticsService.js`.

Mathematical Formulation

Best-of-N CT Averaging:

$$\text{CT Score} = \frac{\overline{\text{Top-3}(\text{CT}_1 \ldots \text{CT}_4)}}{\text{CT}_{\max}} \times 20 \tag{3.1}$$

Individual CO Attainment:

$$\text{CO}_k^{\%} = \frac{\sum_{j \in \text{mapped}(CO_k)} \text{score}_j}{\sum_{j \in \text{mapped}(CO_k)} \text{total}_j} \times 100 \tag{3.2}$$

Class-Level CO Attainment:

$$\text{Class-CO}_k = \frac{|\{s : \text{CO}_k^{\%}(s) \geq \text{KPI}\}|}{|\text{enrolled}|} \times 100 \tag{3.3}$$

PO Attainment (averaged from mapped COs):

$$\text{PO}_m = \frac{\sum_{k:CO_k \to PO_m} \text{CO}_k^{\%}}{|\{k : CO_k \to PO_m\}|} \tag{3.4}$$

Core Engine Code

```
// obeEngine.js -- Best-of-N average
const calculateBestOfNAverage = (marksArray, n = 3) => {
  const valid = marksArray.filter(m => typeof m === 'number' && !isNaN(m));
  if (!valid.length) return 0;
  const sorted = [...valid].sort((a, b) => b - a);
  const topN   = sorted.slice(0, n);
```

```
  return topN.reduce((s, v) => s + v, 0) / topN.length;
};

// analyticsService.js -- CO/PO attainment per student
const calculateOBEAttainment = (assessments, markMap, classInstance) => {
  const coTotals = {}, coScores = {};
  assessments.forEach(a => {
    if (!a.mappedCO) return;
    const id = a._id.toString();
    coTotals[a.mappedCO] = (coTotals[a.mappedCO] || 0) + a.totalMarks;
    coScores[a.mappedCO] = (coScores[a.mappedCO] || 0) + (markMap.get(id)||0);
  });

  const coEntries = Object.keys(coTotals).sort().reduce((acc, co) => {
    const pct = coTotals[co] > 0 ? (coScores[co]/coTotals[co])*100 : 0;
    acc[co] = { obtained: round(coScores[co],1), max: coTotals[co],
                percentage: round(pct,0), achieved: pct >= OBE_THRESHOLD };
    return acc;
  }, {});

  // Aggregate COs -> POs via coPoMapping
  const poAcc = {};
  classInstance.coPoMapping.forEach(({ co, po }) => {
    const d = coEntries[co]; if (!d) return;
    po.forEach(p => {
      if (!poAcc[p]) poAcc[p] = { total:0, count:0 };
      poAcc[p].total += d.percentage; poAcc[p].count++;
    });
  });

  const poEntries = Object.keys(poAcc).sort().reduce((acc, po) => {
    acc[po] = { percentage: round(poAcc[po].total/poAcc[po].count, 0) };
    return acc;
  }, {});

  return { coEntries, poEntries };
};
```

Listing 3.8: OBE Calculation Engine Core

Worked Numerical Example

Best-of-3 CT scores: $\{16, 18, 14\}$, average $= 16$.

Table 3.3: Sample Assessment Data for a Theory Course Student

Assessment	CO	PO(s)	Total	Scored
CT-1	CO1	PO1, PO2	20	16
CT-2	CO2	PO2, PO3	20	14
CT-3	CO1	PO1	20	18
CT-4	CO3	PO3	20	12
Final-Q1a	CO1	PO1, PO2	10	8
Final-Q2b	CO2	PO2	10	7

$$\text{CO1} = \frac{16+18+8}{20+20+10} \times 100 = 84\% \quad (\checkmark)$$
$$\text{CO2} = \frac{14+7}{20+10} \times 100 = 70\% \quad (\checkmark)$$
$$\text{CO3} = \frac{12}{20} \times 100 = 60\% \quad (\checkmark)$$

$$\text{PO1} = \frac{84+84}{2} = 84\%, \quad \text{PO2} = \frac{84+70}{2} = 77\%, \quad \text{PO3} = \frac{70+60}{2} = 65\%$$

If 35 of 42 students attain CO1: Class-CO1 $= \frac{35}{42} \times 100 = 83.3\%$.

3.3 Frontend Feature Implementation

3.3.1 Central Admin Panel

The Central Admin Panel provides university-wide oversight and management. Upon login, the Central Admin sees a high-level analytics dashboard displaying total departments, active courses, total teachers, and total students as KPI cards.

Key Features:

- **Department Management:** Full CRUD operations for all university departments, including name, short name, establishment year, and introductory text.
- **Course Directory:** Filterable course catalogue by department and semester, with inline edit/delete capabilities.

- **Teacher Management:** Onboarding form for Host/Guest teachers with department assignment.
- **Bulk Student Upload:** An Excel-like data grid allowing admins to enroll entire batches of students at once. The system auto-generates RUET-format roll numbers (`SeriesDeptCodeXXX`) and institutional email addresses (`ID@student.ruet.ac.bd`).
- **Series Management:** Create and manage academic year (series) records that serve as a grouping key for courses and student cohorts.

3.3.2 Department Admin Panel

Department Admins have scoped access limited to their own department's data. Their panel is the operational hub for semester-level planning.

Key Features:

- **Course Management (Assigning Roles):** The admin selects a Course Code, Series, Section, and Teacher from cascading dropdowns to create a `ClassInstance`, effectively assigning that teacher their course workload for the semester.
- **Guest Teacher Quick-Add:** A streamlined form to onboard temporary faculty without navigating to the Central Admin panel.
- **Course Review Hub:** An analytics dashboard aggregating all submitted Instructor Experience Reports and Student Feedback across running and finished courses. Displays average ratings per evaluation attribute with gap analysis between student-reported and instructor self-assessed scores.
- **Course Advisor Management:** Assign faculty advisors to student series for pastoral care tracking.
- **Departmental Notices:** Publish course-specific or department-wide announcements visible to relevant students.

3.3.3 Teacher Panel

The Teacher Panel is the most feature-rich section of the application, designed to hide OBE complexity behind an intuitive interface.

Course Hub Dashboard

Teachers see a split view of their Running and Finished courses. Clicking a running course opens its dedicated Course Hub — a sub-dashboard showing:

- Quick stats: enrolled students, CTs taken, assignments given, attendance classes held
- Navigation buttons to all course management actions

Attendance Taking

The attendance interface renders a spreadsheet-like grid with students as rows and dates as columns. Teachers click cells to toggle Present/Absent status. The system calculates attendance percentages in real time and automatically maps them to marks using the scale in Equation **??**.

Assessment Management

For Theory courses, teachers create:

- **Class Tests (CT):** Each mapped to a CO. The Best-of-3 logic (Eq. **??**) is applied automatically.
- **Assignments:** Mapped to COs, normalized to 10 marks.
- **Semester Final:** Question-by-question (Q1a, Q1b, Q2a...) with Section A/B designation and individual CO mapping.

For Sessional courses, teachers create Quiz, Performance/Report, Viva, and LabFinal assessments.

Below each assessment creation form, an Excel-like mark entry grid appears. Typing `A` in any cell marks the student as absent (0 marks).

Evaluation Report

The Evaluation Report auto-generates two views:

1. **General Marksheet:** Traditional grade book with raw marks per component and total score.
2. **OBE Attainment Sheet:** CO/PO attainment percentages per student and class-level attainment summary, exportable to Excel and PDF.

CO Management

Teachers can define, edit, and delete Course Outcomes for their class instance, setting the CO label, description, and PO mappings through a dedicated management page.

Student Feedback System

Teachers can view aggregated anonymous feedback from their students. The standardized 6-question Likert-scale form covers:

1. Level of effort students put into the course
2. Course contents met the CLOs
3. Course contents were well structured
4. Course contents were updated and effective
5. Assessment rubrics were fair and clear
6. Assessment tools were appropriate to evaluate CLOs

Instructor Experience Report

At the end of the semester, teachers submit a structured CQI report comprising Likert-scale ratings for course aspects (syllabus currency, teaching-learning facilities, student engagement, assessment appropriateness) and written analysis in four areas: Syllabus & Course Content, Teaching-Learning Methods, Resources, and Assessment Process.

3.3.4 Student Panel

The Student Panel provides a transparent, read-only view of academic progress.

Key Features:

- **Dashboard:** Running and finished courses with average attendance KPI.
- **Marksheet:** Raw scores per assessment component for each enrolled course.
- **OBE Attainment:** Individual CO/PO attainment percentages with achieved/not-achieved indicators.
- **Attendance Tracker:** Date-wise attendance log with percentage and marks.

- **Course Feedback:** Anonymous submission of the 6-question standardized feedback form.
- **University Directory:** Browse all courses (filterable by department/semester) and faculty members.

3.3.5 Common Components

Table 3.4: Shared Frontend Components

Component	Description
`Sidebar.jsx`	Role-aware collapsible navigation sidebar with grouped menu items, active-state highlighting, and dark/light theme toggle
`Layout.jsx`	Root layout wrapper providing sidebar + main content area with responsive grid
`ProtectedRoute.jsx`	RBAC guard component using React Router v7's `Outlet` pattern
`NotificationDropdown.jsx`	Bell icon dropdown showing unread departmental notices with mark-as-read functionality
`ConfirmDialog.jsx`	Reusable modal dialog for destructive action confirmation
`SeriesSelectField.jsx`	Dropdown component fetching and caching available series from the API

The application supports persistent light/dark theme switching via the `ThemeContext` and is installable as a Progressive Web App through a `manifest.json` referencing the RUET logo icon. The sidebar navigation dynamically builds its menu structure from a `sidebarConfig.js` configuration file, making it trivial to add new pages without modifying the sidebar component itself.

Chapter 4

Project Management & Participation

4.1 Development Methodology

The OBE Evaluation and Grading Webapp was developed using an **Agile/Scrum-inspired methodology**, adapted for a six-person academic project team. Rather than adhering to formal two-week Sprints with daily stand-ups, the team adopted a lightweight iterative approach with feature-driven milestones, continuous integration via Git, and frequent peer code reviews conducted through GitHub pull requests.

4.1.1 Agile Principles Applied

1. **Iterative Delivery:** The application was built and deployed in increments, with each iteration delivering a working, testable slice of functionality. The first iteration established the authentication and basic CRUD scaffold; subsequent iterations layered in OBE-specific features.

2. **Working Software Over Documentation:** The team prioritized getting functional features deployed to Netlify early (by the end of Week 2), enabling real-world testing on mobile and desktop devices throughout development.

3. **Continuous Collaboration:** All six team members shared a single GitHub repository under the `azazarfin/obe-webapp-demo` organization. Feature branches were merged via pull requests, with at least one peer review required before merging to `main`.

4. **Responding to Change:** The requirements evolved significantly during development — most notably, the addition of the Instructor Experience Report module, the CO Management page, and the PWA installation feature — all of which were accommodated without disrupting the existing codebase.

4.1.2 Version Control Workflow

The project used Git for version control with the following branch strategy:

- **main:** Production-ready branch. Netlify deploys automatically from this branch.
- **feature/*:** Short-lived branches for individual features, merged via Pull Request after code review.
- **fix/*:** Hotfix branches for critical bug corrections identified during testing.

Commit messages followed the Conventional Commits specification (`feat:`, `fix:`, `refactor:`, `chore:`) to maintain a clean, navigable history.

4.2 Timeline / Gantt Chart

The project commenced on **18 March 2026** (Initial Commit) and reached its final state on **25 April 2026**, spanning approximately **6 weeks**. The development timeline, reconstructed from the Git commit history and augmented with team planning records, is summarized in Table 4.1.

The project schedule is further illustrated in the Gantt chart (Figure 4.1), which visualizes the concurrent development tracks.

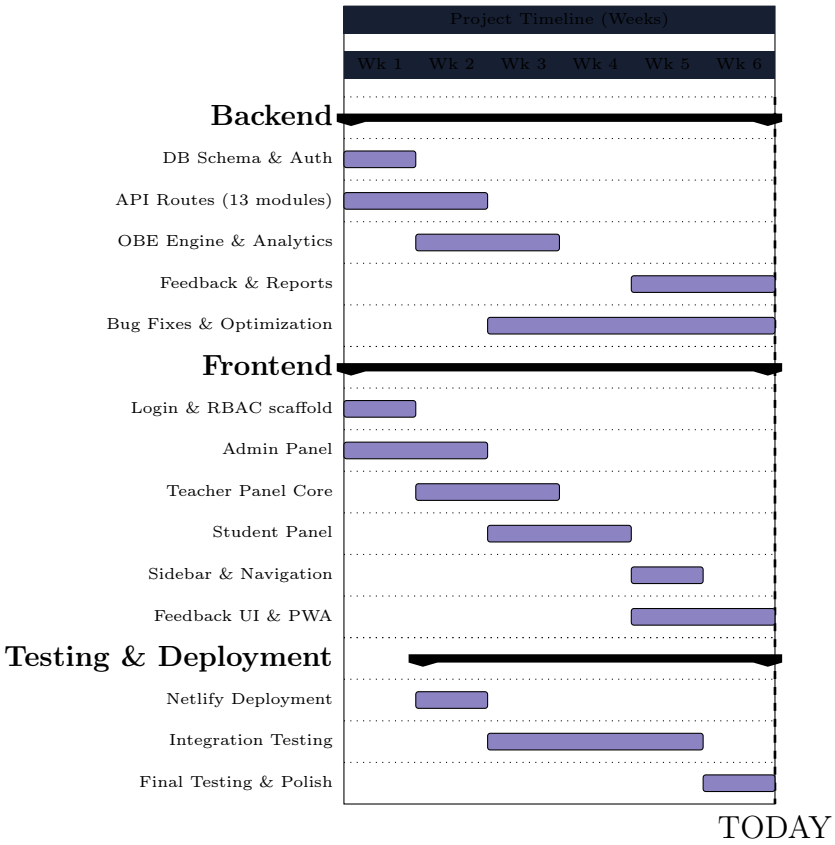

Figure 4.1: Project Development Gantt Chart (Mar 18 – Apr 25, 2026)

Table 4.1: Development Milestone Timeline

Week	Dates	Key Milestone / Deliverable	Git Evidence
1	Mar 18–21	Project setup, database schema design, authentication scaffold, initial backend routes	Initial Commit (56b47b4)
1–2	Mar 19–22	Firebase–JWT integration, CORS configuration, API integration with frontend, major bug fixes	Commits fdb65bb – 85fa759
2	Mar 22	Deployment to Netlify, CORS fixes, logo/redirect fixes	Commits 2523318 – 9fe10d5
3	Mar 24–29	Attendance system redesign, optimization pass, new features (OBE attainment, marksheet)	Commits e080225 – 4352364
4	Mar 30	Major bug fixes, performance optimizations, scalability improvements	Commit 6b8a3ca
5	Apr 19–23	Navigation sidebar, OBE data display (CO/PO marks), theme bug fixes	Commits 0ce7c26 – e77bae5
6	Apr 24	Student feedback system, notification feature	Commits 61894bc – d744543
6	Apr 25	Teacher feedback & report system, PWA install button, CO management page, final code improvements	Commits 86b7f03 – 443879c

4.3 Team Roles & Contributions

The six-member team divided responsibilities across the major technical domains of the project. The contribution split was designed to ensure that each member gained exposure to both frontend and backend development while maintaining clear ownership of specific modules.

4.4 Attendance & Participation

The following table records team attendance at project supervision sessions and collaborative development meetings. Supervisors may fill in the actual records in the provided columns.

Note: P = Present, A = Absent, L = Late. Attendance is recorded by the supervising faculty at each scheduled project progress meeting.

Table 4.2: Team Member Roles and Primary Contributions

Name (Roll No.)	Primary Role	Key Contributions
Md. Azaz Arfin Khan (2310060)	Project Lead & Full-Stack Developer	System architecture design; OBE Calculation Engine; backend API framework; Netlify deployment; GitHub repository management; Teacher Panel (evaluation report, OBE attainment, CO management)
Md. Sanzid Alam Sahil (2310010)	Backend Developer	Database schema design (13 models); authentication system (Firebase + JWT); assessmentRoutes, enrollmentRoutes; bulk student upload logic; instructor report module
Mahathir Mohammad Mahi (2310024)	Frontend Developer	Central Admin Panel (department, course, teacher, student management); bulk upload grid component; series management; dashboard analytics cards
Tahsin Maria (2310034)	Frontend Developer	Student Panel (marksheet, OBE attainment, attendance view, feedback form); university directory; StudentCoursePage; responsive layout adjustments
Mustakim Al Siyam (2310047)	Backend & Integration Developer	feedbackRoutes and analytics aggregation; noticeRoutes; courseAdvisorRoutes; notification service; API integration testing with Postman
Shuvo Chakma (2310061)	UI/UX & QA Developer	Sidebar navigation system; ThemeContext (light/dark mode); PWA manifest; ConfirmDialog; NotificationDropdown; cross-browser compatibility testing; documentation

Table 4.3: Contribution Distribution by System Component (%)

Component	**Azaz**	**Sahil**	**Mahi**	**Tahsin**	**Siyam**	**Shuvo**
Database Design	20	40	10	10	10	10
Backend API	35	30	5	5	20	5
OBE Engine	60	20	5	5	10	–
Admin Panel (Frontend)	10	–	70	–	10	10
Teacher Panel (Frontend)	60	5	5	10	10	10
Student Panel (Frontend)	10	–	10	65	5	10
Feedback System	15	10	–	15	50	10
UI/UX & Components	15	–	15	10	5	55
Testing & QA	20	15	15	15	15	20
Deployment	50	20	10	5	10	5

Table 4.4: Project Meeting Attendance Record

Session No.	**Date**	**Sahil (2310010)**	**Mahi (2310024)**	**Tahsin (2310034)**	**Siyam (2310047)**	**Azaz (2310060)**	**Shuvo (2310061)**	**Remarks**
1								
2								
3								
4								
5								
6								
7								
Supervisor Signature								

References

[1] William G. Spady. "Outcome-Based Education: Critical Issues and Answers". In: *American Association of School Administrators* (1994).

[2] International Engineering Alliance. "Washington Accord: Graduate Attributes and Professional Competencies". In: (2014). Accessed: April 2026. URL: https://www.ieagreements.org/accords/washington/.

[3] Board of Accreditation for Engineering and Technical Education. "BAETE Accreditation Manual for Undergraduate Engineering Programs". In: (2017). Institution of Engineers, Bangladesh.

[4] University Grants Commission of Bangladesh. "Self-Assessment Manual for Public Universities". In: (2023). Institutional Quality Assurance Cell Guidelines.

[5] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 8th. McGraw-Hill Education, 2014. ISBN: 978-0078022128.

[6] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2016. ISBN: 978-0133943030.

[7] Meta Platforms, Inc. *React — A JavaScript Library for Building User Interfaces*. Version 19. Accessed: April 2026. 2024. URL: https://react.dev/.

[8] OpenJS Foundation. *Node.js — JavaScript Runtime*. Accessed: April 2026. 2024. URL: https://nodejs.org/.

[9] OpenJS Foundation. *Express — Fast, Unopinionated, Minimalist Web Framework for Node.js*. Version 5. Accessed: April 2026. 2024. URL: https://expressjs.com/.

[10] MongoDB, Inc. *MongoDB — The Developer Data Platform*. Accessed: April 2026. 2024. URL: https://www.mongodb.com/.

[11] Mongoose Contributors. *Mongoose — Elegant MongoDB Object Modeling for Node.js*. Version 9. Accessed: April 2026. 2024. URL: https://mongoosejs.com/.

[12] Google LLC. *Firebase Authentication*. Accessed: April 2026. 2024. URL: https://firebase.google.com/docs/auth.

[13] Auth0. *JSON Web Tokens — Introduction*. Accessed: April 2026. 2024. URL: https://jwt.io/introduction.

[14] Redux Team. *Redux Toolkit — The Official, Opinionated, Batteries-Included Toolset for Efficient Redux Development*. Version 2. Accessed: April 2026. 2024. URL: https://redux-toolkit.js.org/.

[15] Tailwind Labs. *Tailwind CSS — A Utility-First CSS Framework*. Version 4. Accessed: April 2026. 2024. URL: https://tailwindcss.com/.

[16] Evan You. *Vite — Next Generation Frontend Tooling*. Version 6. Accessed: April 2026. 2024. URL: https://vite.dev/.

[17] Remix Software. *React Router — Declarative Routing for React*. Version 7. Accessed: April 2026. 2024. URL: https://reactrouter.com/.

[18] Lucide Contributors. *Lucide — Beautiful & Consistent Icons*. Accessed: April 2026. 2024. URL: https://lucide.dev/.

[19] Software Freedom Conservancy. *Git — Distributed Version Control System*. Accessed: April 2026. 2024. URL: https://git-scm.com/.

[20] Netlify, Inc. *Netlify — Develop & Deploy the Best Web Experiences*. Accessed: April 2026. 2024. URL: https://www.netlify.com/.

Appendices

Appendix A

Source Code & GitHub Repository

The complete source code for the OBE Evaluation and Grading Webapp is publicly available on GitHub:

https://github.com/azazarfin/obe-webapp-demo

The repository is organized as a monorepo with two top-level directories: `backend/` (Node.js/Express API) and `frontend/` (React/Vite SPA). The following subsections present selected, well-commented source code listings representing the most technically significant modules of the system.

A.1 OBE Engine — obeEngine.js

```
/**
 * OBE Engine Utilities
 * Implements the core business logic for Outcome-Based Education
 * attainment calculations at RUET.
 */

/**
 * Calculates the average of the Best N marks from an array.
 * Used for Theory courses: Best 3 of 4 Class Tests -> 20 marks.
 *
 * @param {number[]} marksArray - Obtained CT marks, e.g., [15,12,18,10]
 * @param {number}   n          - Number of top marks to consider (default 3)
 * @returns {number} Average of best N marks
 */
const calculateBestOfNAverage = (marksArray, n = 3) => {
  if (!marksArray || !Array.isArray(marksArray) || !marksArray.length) {
    return 0;
  }

  // Filter out NaN/non-numeric values defensively
  const validMarks = marksArray.filter(
    m => typeof m === 'number' && !isNaN(m)
  );
```

```
  if (!validMarks.length) return 0;

  // Sort descending and take top N
  const sorted = [...validMarks].sort((a, b) => b - a);
  const topN   = sorted.slice(0, n);

  return topN.reduce((acc, val) => acc + val, 0) / topN.length;
};

module.exports = { calculateBestOfNAverage };
```

Listing A.1: OBE Engine Core: Best-of-N Averaging (backend/services/obeEngine.js)

A.2 Authentication Middleware — authMiddleware.js

```
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET;

// Fail-fast: crash at startup if JWT_SECRET is missing
if (!JWT_SECRET) {
  console.error('FATAL: JWT_SECRET not set. Exiting.');
  process.exit(1);
}

/**
 * Middleware: verify Bearer JWT from Authorization header.
 * Attaches decoded { uid, role, email, userId } to req.user.
 */
const verifyToken = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Unauthorized: No token provided' });
  }

  try {
    req.user = jwt.verify(authHeader.split(' ')[1], JWT_SECRET);
    next();
  } catch {
    return res.status(401).json({ error: 'Unauthorized: Invalid token' });
  }
};

```

```
/**
 * Middleware factory: restrict access to specific roles.
 * Usage: requireRole('TEACHER', 'DEPT_ADMIN')
 */
const requireRole = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({ error: 'Forbidden: Insufficient permissions'
    });
  }
  next();
};

module.exports = { verifyToken, requireRole, JWT_SECRET };
```

Listing A.2: JWT Authentication & Role Guard Middleware

A.3 Enrollment Sync — Automatic Student Registration

```
/**
 * Ensures every eligible student (matching department + series + section)
 * is enrolled in the given ClassInstance. Idempotent -- safe to call
 * on every page load without creating duplicate enrollments.
 */
const syncRegularEnrollmentsForClassInstance = async (classInstance) => {
  const course = classInstance.course;
  if (!course?.department) return 0;

  const filter = {
    role: 'STUDENT',
    department: course.department._id || course.department,
    series: classInstance.series
  };

  // For N/A sections, enroll students who have no section assigned
  if (classInstance.section && classInstance.section !== 'N/A') {
    filter.section = classInstance.section;
  } else {
    filter.$or = [
      { section: 'N/A' }, { section: null },
      { section: '' },    { section: { $exists: false } }
    ];
  }
```

```

  const [students, existing] = await Promise.all([
    User.find(filter).select('_id'),
    Enrollment.find({ classInstance: classInstance._id }).select('student')
  ]);

  const existingIds = new Set(existing.map(e => e.student.toString()));
  const toInsert = students
    .filter(s => !existingIds.has(s._id.toString()))
    .map(s => ({ student: s._id, classInstance: classInstance._id,
                 type: 'regular', status: 'active' }));

  if (toInsert.length > 0) await Enrollment.insertMany(toInsert);
  return toInsert.length;
};
```

Listing A.3: Auto-Sync Regular Enrollments for a ClassInstance

A.4 Feedback Routes — Aggregated Analytics

```
// POST /api/feedback/:classInstanceId
// Students submit their anonymous 6-question Likert feedback
router.post('/:classInstanceId',
  verifyToken,
  requireRole('STUDENT'),
  async (req, res) => {
    const { classInstanceId } = req.params;
    const { ratings } = req.body;   // Array of { question, rating (1-5) }

    // Validate all 6 questions answered
    if (!ratings || ratings.length !== 6) {
      return res.status(400).json({ error: 'All 6 questions must be answered' })
    ;
    }

    const classInstance = await ClassInstance.findById(classInstanceId);
    if (!classInstance?.feedbackPublished) {
      return res.status(403).json({ error: 'Feedback window is not open' });
    }

    const feedback = new Feedback({
      classInstance: classInstanceId,
      ratings,
      submittedAt: new Date()
```

```
      // Note: no student ID stored -- anonymous by design
    });

    await feedback.save();
    res.status(201).json({ message: 'Feedback submitted successfully' });
  }
);
```

Listing A.4: Student Feedback Submission Endpoint

A.5 Frontend App Entry — App.jsx

```
function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <BrowserRouter>
          <Routes>
            <Route path="/login" element={<Login />} />
            <Route path="/unauthorized" element={<Unauthorized />} />

            <Route path="/" element={<Layout />}>
              {/* Central Admin */}
              <Route element={<ProtectedRoute allowedRoles={['CENTRAL_ADMIN']}/>}>
                <Route path="central-admin" element={<CentralAdminDashboard />}/>
              </Route>

              {/* Department Admin */}
              <Route element={<ProtectedRoute allowedRoles={['DEPT_ADMIN']}/>}>
                <Route path="dept-admin" element={<DeptAdminDashboard />}/>
              </Route>

              {/* Teacher */}
              <Route element={<ProtectedRoute allowedRoles={['TEACHER']}/>}>
                <Route path="teacher" element={<TeacherDashboard />}/>
              </Route>

              {/* Student */}
              <Route element={<ProtectedRoute allowedRoles={['STUDENT']}/>}>
                <Route path="student" element={<StudentDashboard />}/>
              </Route>

```

```
              <Route index element={<HomeRedirect />} />
            </Route>
          </Routes>
        </BrowserRouter>
      </AuthProvider>
    </ThemeProvider>
  );
}
```

Listing A.5: React Application Root with RBAC Routing (frontend/src/App.jsx)

Appendix B

Application Screenshots

This appendix presents screenshots captured from the live deployment of the OBE Evaluation and Grading Webapp (https://obe-webapp-demo.netlify.app). The screenshots demonstrate the key interfaces and features of all four user panels.

B.1 Login Page

The login page provides Firebase-authenticated email/password entry with RUET institutional branding. All users — Central Admins, Department Admins, Teachers, and Students — share a single login endpoint, with the system automatically routing them to their role-appropriate dashboard upon successful authentication.

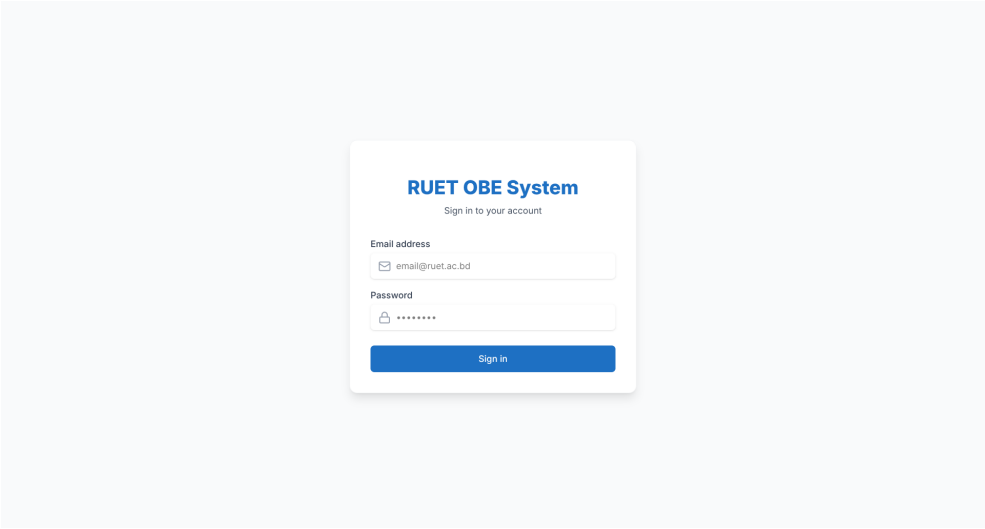

Figure B.1: Login Page — RUET OBE Evaluation System

B.2 Central Admin Panel

The Central Admin dashboard displays aggregate university metrics (Total Departments, Total Courses, Total Teachers, Total Students) as KPI cards, followed by navigation tiles for Department Info, Course Info, Teacher Info, Student Info, and Series Management.

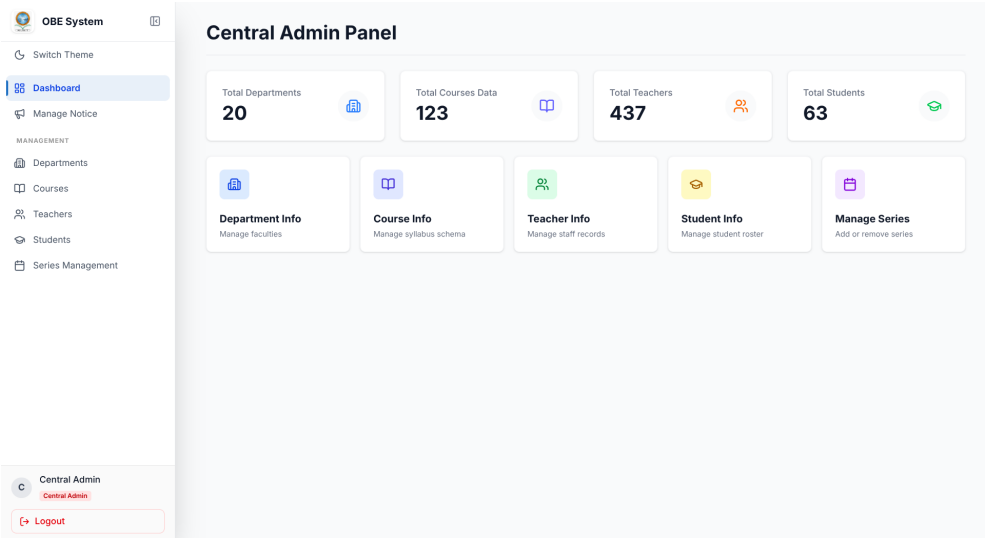

Figure B.2: Central Admin Dashboard — University Overview

B.3 Department Admin Panel

The Department Admin dashboard shows the Feedback Analytics hub — a unified view comparing student satisfaction ratings with instructor self-ratings across thematic categories (Syllabus & Content, Teaching & Learning, Resources, Assessment). The sidebar provides navigation to Course Management, Add/Edit Courses, Teachers, Students, Course Advisors, and Feedback Analytics.

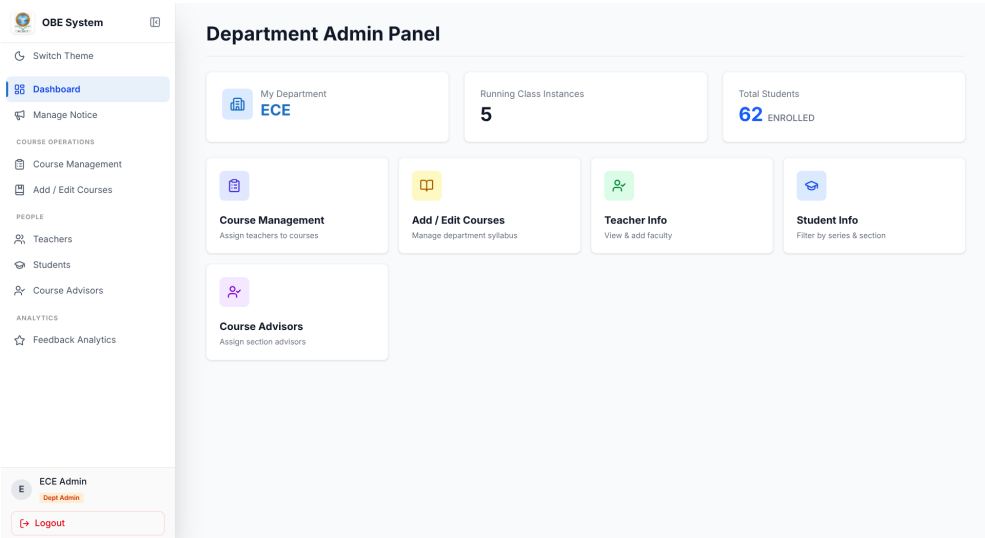

Figure B.3: Department Admin — Feedback Analytics Dashboard

B.4 Teacher Dashboard

The Teacher Dashboard displays the logged-in teacher's profile with Running Courses and Finished Courses sections. Each course card shows the course code, name, type (Theory/Sessional), series, section, and current status.

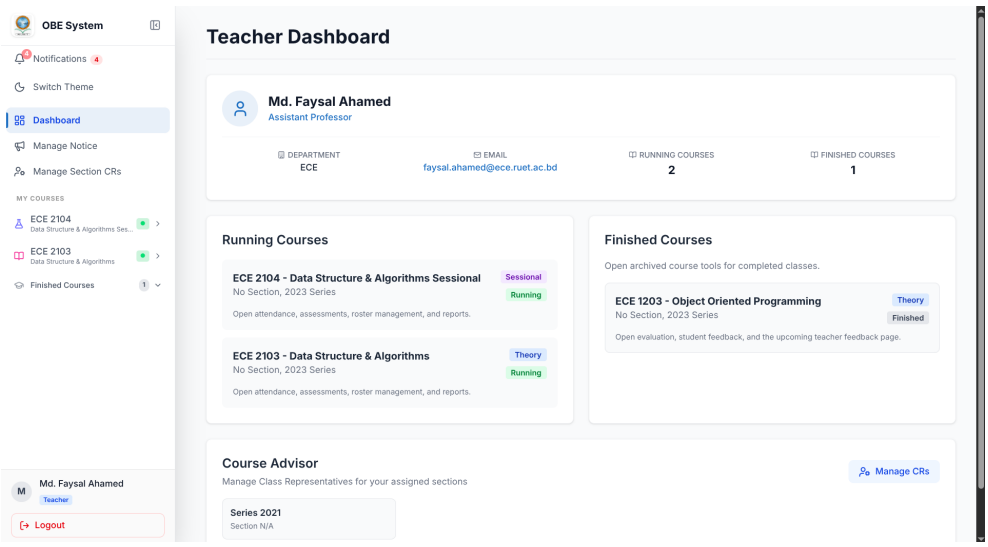

Figure B.4: Teacher Dashboard — Course Overview

B.5 Teacher Course Hub

The Course Hub displays KPI cards (Students, COs Defined, Quizzes, Attendance) and Quick Action tiles for Take Attendance, Add Assessment, Manage COs, Manage Notice, Manage Assessments, Modify Student Roster, Evaluation Report, and Student Feedback.

B.6 Attendance Taking Interface

The attendance interface displays a spreadsheet-like grid with student IDs and names as rows, class dates as columns, and P (Present, green) / A (Absent, red) status indicators in each cell. The rightmost column shows the cumulative attendance percentage per student. Action buttons include "+ Add Class Date" and "Save Records."

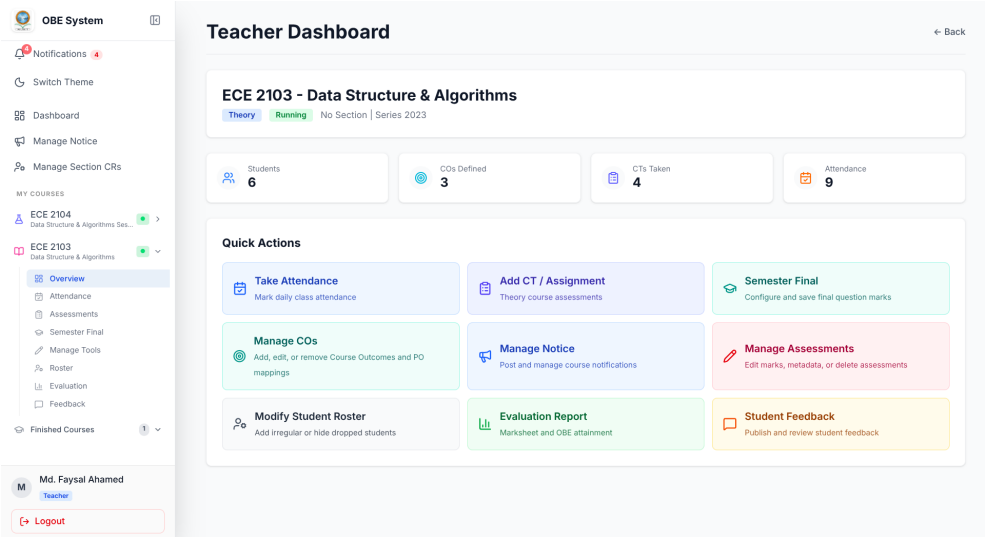

Figure B.5: Teacher Course Hub — Quick Actions

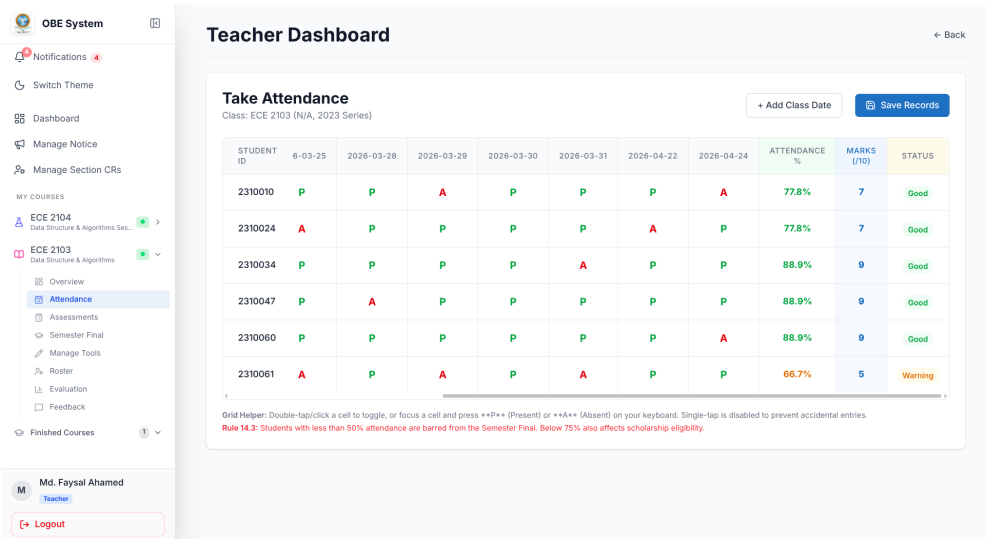

Figure B.6: Attendance Taking Interface — Spreadsheet-Style Grid

B.7 Evaluation Report — General Marksheet

The Evaluation Report displays two tabs: General Marksheet and OBE Attainment Profile. The General Marksheet shows component-wise marks per student, with auto-computed Total and Grade columns. Export buttons for Excel and PDF are available.

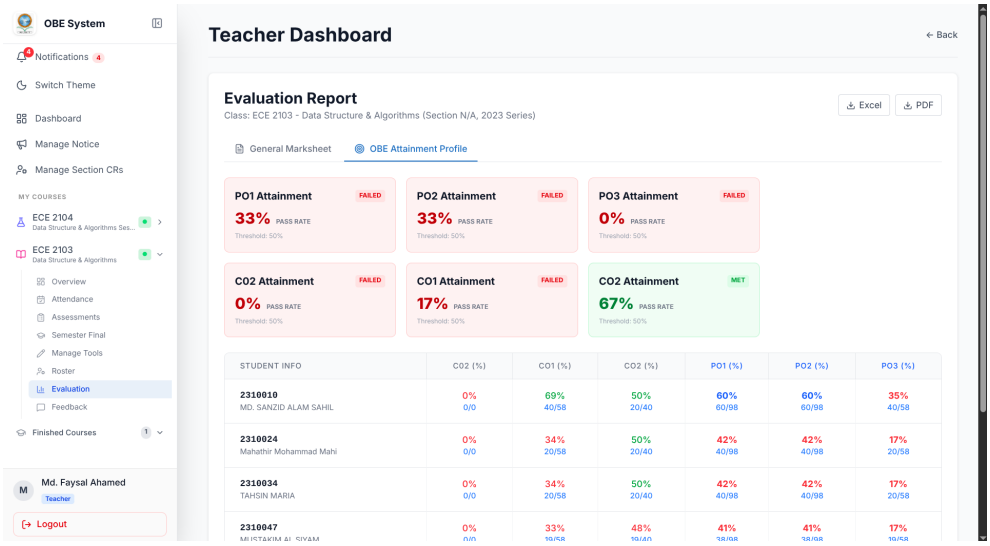

Figure B.7: Evaluation Report — General Marksheet with Grade Computation

B.8 Student Dashboard

The Student Portal displays the student's profile, academic statistics, course cards with per-course attendance percentages, and a Program Outcomes sidebar showing PO attainment bars. The sidebar lists all enrolled courses with expandable sub-navigation.

B.9 Student Course Page

The Student Course Page shows attendance, marks so far, CO attainment, course type, and resource navigation cards for Attendance Info, Marksheet, OBE Attainment Info, and Give Feedback.

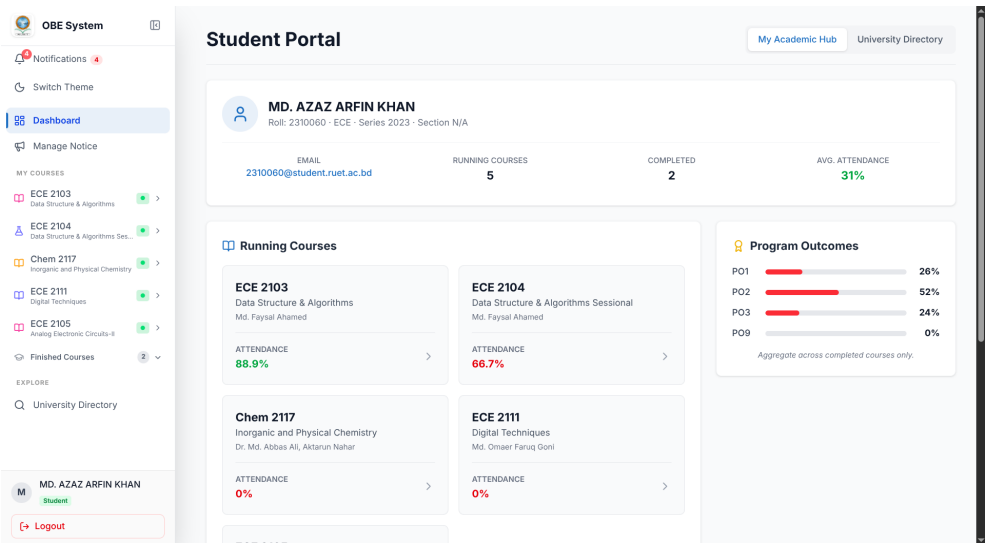

Figure B.8: Student Portal — Dashboard with Running Courses and PO Attainment

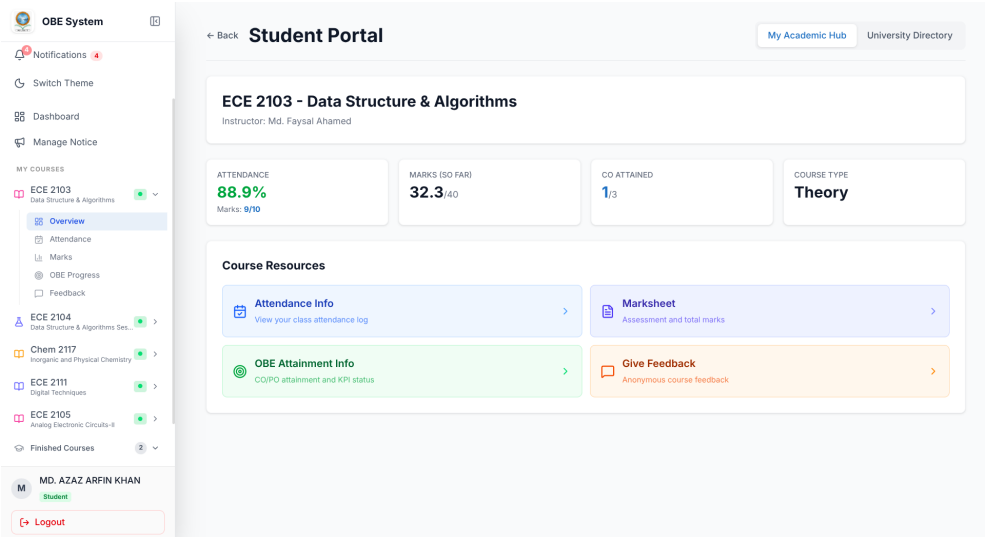

Figure B.9: Student Course Page — Overview with Resource Cards

Appendix C

User Manual

This appendix provides a concise operational guide for each user role within the OBE Evaluation and Grading Webapp.

C.1 Getting Started

1. Open a web browser (Chrome 90+, Firefox 88+, Safari 14+, or Edge 90+).
2. Navigate to the application URL: https://obe-webapp-demo.netlify.app
3. Enter your institutional email address and password.
4. Click **"Sign In"**. The system will automatically redirect you to your role-appropriate dashboard.
5. To install as a Progressive Web App (PWA), click the **"Install"** button in your browser's address bar or use the browser's "Add to Home Screen" option.

C.2 Central Admin Operations

C.3 Department Admin Operations

C.4 Teacher Operations

C.5 Student Operations

C.6 Troubleshooting

Table C.1: Central Admin Feature Guide

Action	**Steps**
Add a Department	Click "Department Info" → "Add Department" → Fill name, short name, year → Submit
Add a Course	Click "Course Info" → "Add Course" → Select department, type, semester, fill code/name → Submit
Add a Teacher	Click "Teacher Info" → "Add Teacher" → Fill name, email, designation, department, type → Submit
Bulk Upload Students	Click "Student Info" → Select department and series → Click "+ Add" to add rows → Fill names → Click "Save"
Manage Series	Click "Series Management" → "Create Series" → Enter year (e.g., 2023) → Submit

Table C.2: Department Admin Feature Guide

Action	**Steps**
Assign Teacher to Course	Sidebar → "Course Management" → Select Course Code, Series, Section, Teacher → "Assign"
Add Guest Teacher	Sidebar → "Teachers" → "Add Guest Teacher" → Fill details → Submit
View Feedback Analytics	Sidebar → "Feedback Analytics" → View KPI cards, gap analysis, per-course feedback
Assign Course Advisor	Sidebar → "Course Advisors" → Select teacher and series → "Assign"
Publish Notice	Sidebar → "Manage Notice" → "Create Notice" → Select course/audience → Write content → Publish

Table C.3: Teacher Feature Guide

Action	**Steps**
Take Attendance	Dashboard → Click course card → "Take Attendance" → Click "+ Add Class Date" → Toggle P/A for each student → "Save Records"
Create Assessment	Course Hub → "Add Assessment" → Select type (CT/Assignment/Final/Quiz) → Map to CO → Set total marks → Save → Enter marks in grid
Enter Marks	After creating assessment, a spreadsheet grid appears → Enter scores per student → Type 'A' for absent students → Click "Save"
Manage COs	Course Hub → "Manage COs" → Add/edit/delete COs → Map each CO to POs → Add descriptions
View Evaluation Report	Course Hub → "Evaluation Report" → Toggle "General Marksheet" / "OBE Attainment Profile" tabs → Export via "Excel" or "PDF" buttons
Submit Instructor Report	Course Hub → "Instructor Experience Report" → Rate each category (1–5) → Write analysis → Submit
Publish Feedback	Course Hub → "Student Feedback" → Click "Publish Feedback" to open the feedback window for students
Manage Assessments	Course Hub → "Manage Assessments" → Edit marks, update metadata, or delete assessments

Table C.4: Student Feature Guide

Action	Steps
View Dashboard	After login, view running courses with attendance percentages and PO attainment bars
View Course Details	Click any course card or sidebar course name → See attendance, marks, CO attainment, course type
View Attendance	Course page → "Attendance Info" → View date-wise attendance log with percentage
View Marks	Course page → "Marksheet" → View per-component marks and total
View OBE Attainment	Course page → "OBE Attainment Info" → View CO-wise and PO-wise attainment percentages
Submit Feedback	Course page → "Give Feedback" → Rate 6 questions on a Likert scale (1–5) → Submit (anonymous)
Browse Directory	Sidebar → "University Directory" → Filter by department/semester → Browse courses and teachers
Check Notifications	Top sidebar → "Notifications" bell icon → View unread notices → Click to mark as read

Table C.5: Common Issues and Solutions

Issue	Solution
Login fails	Verify email format (e.g., `2310060@student.ruet.ac.bd`). Check caps lock. Contact admin if account is not created.
Page not loading	Clear browser cache. Check internet connection. Try a different browser.
Marks not saving	Ensure all fields are numeric (use 'A' for absent only). Check internet connectivity.
Attendance not updating	Click "Save Records" after toggling statuses. Ensure at least one class date exists.
Feedback form disabled	The teacher must first "Publish Feedback" from their panel before students can submit.
Wrong dashboard	Your role determines your panel. Contact your Department Admin to verify your account role.